Université Moulay Ismaïl
Faculté Polydisciplinaire d'Errachidia

Support de cours avec des exemples corrigés

Programmation en langage C

Licence Digitalisation

Tronc Commun Informatique Appliquée – Semestre 1

Année universitaire : 2024/2025

Pr. Mounir HANINE

Table des matières

Chapitre 0 : Architecture de l'ordinateur et systèmes de numération.....5

0.1 Architecture et composants de l'ordinateur.....5

0.2 Exécution d'un programme.....6

0.3 Pourquoi étudier les systèmes de numération ?.....6

0.4 Les principaux systèmes de numération.....7

0.5 Conversions entre bases.....8

0.6 Représentation des nombres en mémoire.....8

0.7 Représentation du texte.....11

0.8 Opérations binaires.....13

Chapitre 1 : Introduction au langage C.....15

1.1 Qu'est-ce que le langage C ?.....15

1.2 Structure d'un programme C.....15

1.3 Le processus de compilation.....16

1.4 Configurer l'environnement de travail.....17

1.5 Les commentaires.....17

1.6 Premier programme.....18

1.7 Les séquences d'échappement.....18

1.8 Les types de base.....19

1.9 La déclaration des variables.....21

Chapitre 2 : Les opérateurs standards.....22

2.1 L'affectation.....22

2.2 Les opérateurs arithmétiques.....22

2.3 Les opérateurs logiques.....22

2.4 Les opérateurs de comparaison.....23

2.5 L'incrémentation et la décrémentation.....23

2.6 Les priorités des opérateurs.....24

Chapitre 3 : Les conversions de type.....25

3.1 Les conversions automatiques.....25

3.2 Les conversions lors de l'appel d'une fonction.....25

3.3 Une erreur classique.....25

3.4 Les règles de conversion dans une opération.....26

3.5 La conversion forcée....26

Chapitre 4 : Lire et écrire des données....27

4.1 L'écriture formatée avec printf....27

4.2 La lecture formatée avec scanf....29

4.3 Lire et écrire un caractère....30

4.4 Ecrire un caractère dans un fichier....31

Chapitre 5 : Les structures alternatives....32

5.1 L'instruction if....32

5.2 L'instruction if...else....32

5.3 Les tests successifs....33

5.4 A quel if se rattache un else ?....33

5.5 L'instruction switch....35

5.6 Exercice d'application....36

Chapitre 6 : Les structures répétitives....38

6.1 La boucle while....38

6.2 La boucle do...while....39

6.3 La boucle for....40

6.4 Quelle boucle choisir ?....40

Chapitre 7 : Les tableaux....41

7.1 Les tableaux à une dimension....41

7.2 Les tableaux à deux dimensions....42

7.3 Application....43

Chapitre 8 : Les chaînes de caractères....45

8.1 Déclaration et initialisation....45

8.2 Les chaînes constantes....45

8.3 La représentation en mémoire....46

8.4 L'accès aux éléments....47

8.5 Les fonctions de stdio.h....47

8.6 Les fonctions de string.h....49

8.7 Les conversions entre chaînes et nombres....50

8.8 Les tableaux de chaînes....51

Chapitre 9 : Les fonctions.....53
9.1 La déclaration..... 53
9.2 L'implémentation.....53
9.3 L'appel d'une fonction.....54
9.4 La portée des variables.....55
9.5 Les règles à respecter.....56
9.6 Exercice d'application.....57

Chapitre 0 : Architecture de l'ordinateur et systèmes de numération

Avant d'écrire un programme, il est utile de comprendre la machine qui va l'exécuter. Ce chapitre présente les composants d'un ordinateur, le déroulement d'un programme, puis la façon dont les nombres et les textes sont représentés en mémoire.

0.1 Architecture et composants de l'ordinateur

Un système informatique se compose de deux grandes parties. La partie matérielle, ou **hardware**, regroupe les composants physiques de l'appareil. La partie logicielle, ou **software**, comprend les programmes, les applications et le système d'exploitation, qui permettent d'utiliser le matériel pour accomplir les tâches demandées.

Les périphériques

Les périphériques entourent l'ordinateur et se répartissent en deux grandes catégories. Les **périphériques d'entrée** fournissent des informations au système : c'est le cas de la souris, du clavier ou d'une manette de jeu. Les **périphériques de sortie** restituent des informations : c'est le cas de l'écran et de l'imprimante.

Le processeur

Le processeur, ou CPU (Central Processing Unit), est le cerveau de l'ordinateur. Il traite les instructions des programmes, les décode et réalise les calculs nécessaires pour produire les résultats attendus.

La mémoire vive

La mémoire vive, ou RAM (Random Access Memory), est une mémoire temporaire. Elle contient les données dont le processeur a besoin pour exécuter les programmes en cours, ce qui lui évite d'accéder sans cesse au disque dur, beaucoup plus lent.

Le disque dur

Le disque dur conserve les données de manière permanente, même lorsque l'ordinateur est éteint. Il contient les fichiers, les programmes et le système d'exploitation. Au démarrage et à l'ouverture d'un programme, les informations nécessaires sont chargées du disque vers la RAM afin que le processeur puisse y accéder rapidement.

Il existe deux grandes familles de disques. Les **HDD** (Hard Disk Drive) utilisent des plateaux magnétiques : ils offrent une grande capacité à un coût inférieur, mais ils sont plus lents et sensibles aux chocs. Les **SSD** (Solid State Drive) utilisent de la mémoire flash : ils sont beaucoup plus rapides et plus résistants, mais coûtent généralement plus cher par gigaoctet.

La carte graphique

La carte graphique, ou GPU (Graphics Processing Unit), décharge le processeur de certains calculs, en particulier ceux liés à la 3D et au traitement d'images. Un GPU dédié est surtout utile pour les jeux vidéo, le montage vidéo ou la modélisation 3D. Pour la bureautique et la navigation sur le Web, le processeur avec son GPU intégré suffit généralement.

Le BIOS

Le BIOS est un composant essentiel au démarrage de l'ordinateur. Il contient un programme qui initialise les composants matériels et charge le système d'exploitation.

La carte mère

La carte mère relie tous les composants essentiels : le processeur, la mémoire vive, le disque dur, ainsi que les éventuelles cartes graphiques ou sonores. Elle assure leur alimentation et leur communication grâce à des circuits appelés **bus**.

0.2 Exécution d'un programme

L'exécution d'un programme suit les étapes suivantes :

1. Le programme est chargé du disque dur vers la mémoire RAM.
2. Le processeur récupère les instructions, les décode et effectue les opérations demandées.
3. Les résultats sont stockés temporairement en RAM, puis affichés à l'écran par les circuits de sortie.

Toutes ces opérations se font en binaire : chaque donnée et chaque instruction est représentée par une suite de 0 et de 1.

0.3 Pourquoi étudier les systèmes de numération ?

Les ordinateurs ne manipulent que le langage binaire. Qu'il s'agisse de texte, d'image ou de son, toute donnée est convertie en binaire pour être traitée. En langage C, on rencontre souvent le binaire, l'octal et l'hexadécimal, car ces bases facilitent la manipulation des données et aident à comprendre le fonctionnement des algorithmes.

La gestion de la mémoire et l'optimisation du code demandent parfois d'opérer directement sur les bits, par exemple pour des options ou des permissions de fichiers. De même, les caractères sont représentés par des codes numériques, comme l'ASCII.

0.4 Les principaux systèmes de numération

Le système décimal (base 10)

C'est le système que nous utilisons au quotidien. Il emploie les chiffres de 0 à 9, et chaque position correspond à une puissance de 10 :

$$375 = (3 \times 10^2) + (7 \times 10^1) + (5 \times 10^0)$$

Le système binaire (base 2)

Le binaire est utilisé par les ordinateurs, car ils ne manipulent que deux états. Il emploie les chiffres 0 et 1, appelés **bits**, et chaque position correspond à une puissance de 2 :

$$(1011)_2 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 11 \text{ en décimal}$$

Le système octal (base 8)

L'octal emploie les chiffres de 0 à 7, et chaque position correspond à une puissance de 8. Il sert notamment pour les permissions de fichiers sous UNIX. Chaque chiffre octal correspond à 3 bits.

$$(17)_8 = (1 \times 8^1) + (7 \times 8^0) = 15 \text{ en décimal}$$

$$7 \text{ en octal} = 111 \text{ en binaire}$$

Dans une permission comme 754, les trois chiffres concernent successivement le propriétaire, le groupe et les autres utilisateurs. Chaque chiffre additionne les droits de lecture (r = 4), d'écriture (w = 2) et d'exécution (x = 1). Ainsi, 754 signifie : tous les droits pour le propriétaire, lecture et exécution pour le groupe, et lecture seule pour les autres.

Le système hexadécimal (base 16)

L'hexadécimal représente de grandes valeurs de façon compacte, comme les adresses mémoire. Il emploie les chiffres de 0 à 9 et les lettres de A à F, avec A = 10, B = 11, C = 12, D = 13, E = 14 et F = 15. Chaque chiffre hexadécimal correspond à 4 bits.

```
(2F)16 = (2 × 16¹) + (15 × 16⁰) = 47 en décimal
(2F)16 = 0010 1111 en binaire
```

0.5 Conversions entre bases

Du binaire vers le décimal

On multiplie chaque bit par la puissance de 2 correspondante, puis on additionne les résultats. Comme on l'a vu, $(1011)_2$ vaut 11 en décimal.

Du décimal vers le binaire

On divise le nombre par 2 en conservant les restes, jusqu'à obtenir un quotient nul. Le résultat se lit ensuite de bas en haut.

```
13 ÷ 2 = 6   reste 1
 6 ÷ 2 = 3   reste 0
 3 ÷ 2 = 1   reste 1
 1 ÷ 2 = 0   reste 1

Résultat : (13)10 = (1101)2
```

Du binaire vers l'hexadécimal

On groupe les bits par 4, de droite à gauche, puis on remplace chaque groupe par son équivalent hexadécimal. Ainsi, $(11100110)_2$ s'écrit 1110 0110, soit $(E6)_{16}$.

Du décimal vers l'hexadécimal

On divise le nombre par 16 jusqu'à obtenir un quotient nul, puis on lit les restes de bas en haut.

47 ÷ 16 = 2 reste 15, soit F
2 ÷ 16 = 0 reste 2
Résultat : $(47)_{10} = (2F)_{16}$

255 ÷ 16 = 15 reste 15, soit F
15 ÷ 16 = 0 reste 15, soit F
Résultat : $(255)_{10} = (FF)_{16}$

De la même façon : $(100)_{10} = (64)_{16}$

0.6 Représentation des nombres en mémoire

Les entiers positifs

Un entier positif est directement représenté en binaire. Comme $5 = 2^2 + 2^0$, on a $5 = (101)_2$.

Sur 8 bits, on le complète avec des zéros à gauche : 00000101. Avec n bits, on peut représenter les valeurs de 0 à $2^n - 1$. Sur 8 bits, la plage va donc de 0 à 255.

Les entiers négatifs

Pour représenter les nombres négatifs, on utilise des méthodes comme le complément à un et le complément à deux.

Le **complément à un** consiste à inverser tous les bits. Pour +5 = 00000101, on obtient 11111010.

Le **complément à deux** consiste à prendre le complément à un, puis à ajouter 1. C'est la méthode la plus courante. On obtient ainsi -5 :

```
+5 : 00000101
Complément à un : 11111010
On ajoute 1 : 11111011   qui représente -5
```

Avec n bits, le complément à deux permet de représenter les valeurs de -2^{n-1} à $2^{n-1} - 1$. Sur 8 bits, la plage va de -128 à 127.

Les nombres à virgule flottante

Les nombres réels sont représentés selon la norme **IEEE 754**, sous la forme générale :

```
Valeur = signe × mantisse × 2^exposant
```

La mantisse contient les chiffres significatifs du nombre, tandis que l'exposant en fixe l'échelle. Les deux formats les plus courants sont les suivants :

Format	Taille	Signe	Exposant	Mantisse	Décalage de l'exposant
Simple précision	32 bits	1 bit	8 bits	23 bits	127
Double précision	64 bits	1 bit	11 bits	52 bits	1023

Le bit de signe vaut 0 pour un nombre positif et 1 pour un nombre négatif. La mantisse enregistrée correspond aux bits situés après la virgule, une fois le nombre écrit sous la forme $1,xxx \times 2^e$.

Exemple 1 : représenter -5,75 en simple précision

1. On code la valeur absolue en binaire : 5,75 = 101,11.
2. On normalise : $101,11 = 1,0111 \times 2^2$.
3. La mantisse sur 23 bits est : 0111 0000 0000 0000 0000 000.

4. L'exposant décalé est 2 + 127 = 129, soit 1000 0001.

5. Le signe vaut 1, car le nombre est négatif.

```
1 10000001 01110000000000000000000
```

Exemple 2 : représenter 12,625 en simple précision

On écrit 12,625 = 1100,101 = 1,100101 × 2^3. L'exposant décalé vaut 3 + 127 = 130 et le signe vaut 0.

```
0100 0001 0100 1010 0000 0000 0000 0000 = (414A0000)16
```

Exemple 3 : représenter 0,375 en simple précision

On multiplie la partie fractionnaire par 2 pour obtenir les bits successifs :

```
0,375 × 2 = 0,75   bit 0
0,75  × 2 = 1,5    bit 1
0,5   × 2 = 1      bit 1
```

On a donc 0,375 = 0,011 = 1,1 × 2^{-2}. L'exposant décalé vaut -2 + 127 = 125, soit 0111 1101.

```
0011 1110 1100 0000 0000 0000 0000 0000 = (3EC00000)16
```

Pour -0,375, seul le bit de signe change. On obtient $(BEC00000)_{16}$.

Exemple 4 : représenter 12,625 en double précision

Le nombre reste 1,100101 × 2^3, mais l'exposant décalé vaut 3 + 1023 = 1026, soit 100 0000 0010. On obtient $(4029400000000000)_{16}$. Pour -12,625, le bit de signe passe à 1 et on obtient $(C029400000000000)_{16}$.

Les valeurs particulières

La norme IEEE 754 réserve certaines combinaisons à des valeurs particulières. En simple précision :

Valeur	Signe	Exposant	Mantisse
+0	0	00000000	Tous les bits à 0
-0	1	00000000	Tous les bits à 0
+∞	0	11111111	Tous les bits à 0
-∞	1	11111111	Tous les bits à 0

Valeur	Signe	Exposant	Mantisse
NaN (Not a Number)	0 ou 1	11111111	Non nulle

0.7 Représentation du texte

L'ordinateur doit aussi manipuler des lettres, des chiffres et des symboles. Ces caractères sont donc codés en binaire selon des standards communs.

Le code ASCII

ASCII (American Standard Code for Information Interchange) a été développé dans les années 1960. Il est normalisé par l'ANSI (American National Standards Institute). Dans sa version de base, il utilise 7 bits, ce qui permet de coder 128 caractères : les lettres anglaises, les chiffres, la ponctuation, quelques symboles et des caractères de contrôle.

Caractères	Codes décimaux
Caractères de contrôle, non affichables	0 à 31
Espace	32
Point d'exclamation	33
Chiffres de 0 à 9	48 à 57
Lettres majuscules de A à Z	65 à 90
Lettres minuscules de a à z	97 à 122

Pour passer d'une majuscule à la minuscule correspondante, on ajoute 32. Par exemple, B a pour code 66 et b pour code 98.

Le mot « Bonjour » est ainsi représenté par les octets suivants :

```
B : 01000010
o : 01101111
n : 01101110
j : 01101010
o : 01101111
u : 01110101
r : 01110010
```

Les principaux caractères de contrôle

Nom	Signification	Action
CR	Carriage return	Retour au début de la ligne
LF	Line feed	Passage à la ligne suivante
BS	Backspace	Retour arrière
HT	Horizontal tabulation	Tabulation horizontale
VT	Vertical tabulation	Tabulation verticale
SP	Space	Espace
FF	Form feed	Passage à la page suivante
DEL	Delete	Suppression
BEL	Bell	Signal sonore

L'ASCII étendu et ses limites

L'ASCII de base a été conçu pour l'anglais et ne contient donc pas les lettres accentuées. Sa version étendue, sur 8 bits, code 256 caractères et ajoute notamment é, à, ç et d'autres symboles. Elle reste toutefois limitée : elle ne couvre pas l'ensemble des écritures du monde, comme l'arabe, le grec ou le cyrillique, ni les émojis.

Unicode et UTF-8

Pour surmonter ces limites, la norme **Unicode** a été publiée en 1991. Elle recense les caractères de tous les systèmes d'écriture, ainsi que les émojis et d'autres symboles modernes. Unicode attribue un numéro à chaque caractère, mais ne fixe pas à lui seul la façon de le stocker. Pour cela, on utilise un encodage comme UTF-8, UTF-16 ou UTF-32.

UTF-8 est le plus utilisé, notamment sur le Web. Il code chaque caractère sur 1 à 4 octets. Il est compatible avec l'ASCII de base : les 128 premiers caractères ont exactement le même code dans les deux systèmes.

0.8 Opérations binaires

Les opérations binaires suivent la même logique que les opérations décimales.

L'addition

Opération	Résultat
0 + 0	0
0 + 1	1
1 + 0	1
1 + 1	10, soit 0 avec une retenue de 1

```
1011 + 1101 = 11000                (11 + 13 = 24)
1110 + 101 = 10011                 (14 + 5 = 19)
110101,1101 + 11,011 = 111001,0011
100011,11 + 101,101 = 101001,011
```

La soustraction

Opération	Résultat
0 - 0	0
1 - 0	1
1 - 1	0
0 - 1	1, avec un emprunt à la position suivante

```
10101 - 1101 = 1000                    (21 - 13 = 8)
10010 - 1101 = 101                     (18 - 13 = 5)
101101,11011 - 11101,111 = 1111,11111
```

La multiplication

La multiplication binaire est directe : 0 × 0, 0 × 1 et 1 × 0 valent 0, tandis que 1 × 1 vaut 1. On pose l'opération comme en décimal, en décalant chaque ligne.

```
      110        (6)
    × 101        (5)
    -----
      110
     000
    110
    -----
    11110        (30)
      111        (7)
    ×  11        (3)
    -----
      111
     111
    -----
    10101        (21)
```

Avec des nombres à virgule, on obtient par exemple : 111000011,11101 × 110,1 = 101101111001,011001.

La division

La division binaire se pose comme une division décimale, en n'utilisant que les chiffres 0 et 1.

```
1111010 ÷ 1011 = 1011, reste 1        (122 ÷ 11 = 11, reste 1)
10001110000 ÷ 111 = 10100010, reste 10 (1136 ÷ 7 = 162, reste 2)
```

Chapitre 1 : Introduction au langage C

1.1 Qu'est-ce que le langage C ?

Le langage C est l'un des langages les plus importants et les plus influents de l'histoire de l'informatique. Il a été développé dans les années 1970 par Dennis Ritchie, chez AT&T Bell Labs, pour la création du système d'exploitation Unix. Il a ensuite servi à écrire de nombreux systèmes d'exploitation, ainsi que des applications qui exigent de bonnes performances et un contrôle précis du matériel.

C'est un langage structuré, mais très proche de la machine. Il est encore largement employé pour les systèmes d'exploitation comme Linux, les compilateurs, les applications embarquées, les applications en temps réel et tous les domaines où la performance et la gestion de la mémoire sont cruciales.

Quelques repères historiques permettent de situer son évolution :

- 1978 : Kernighan et Ritchie publient « The C Programming Language », qui donne la définition classique du langage.
- Années 1980 : l'apparition de compilateurs chez d'autres éditeurs rend nécessaire un standard précis, le C ANSI.
- 1983 : le langage C++, créé chez AT&T, reçoit son nom.
- 1988 : la seconde édition de « The C Programming Language » paraît.
- 1989 : la norme ANSI du langage C est adoptée.

1.2 Structure d'un programme C

Un programme C est composé d'éléments organisés de manière précise, afin que le compilateur puisse le traduire correctement. Voici le programme le plus classique :

```
#include <stdio.h> // Bibliothèque standard

// La fonction main() est le point d'entrée du programme
int main() {
    printf("Hello, World!\n"); // Affiche un message à l'écran
    return 0; // Retourne un statut de réussite
}
```

Résultat à l'exécution :

```
Hello, World!
```

Ce programme contient les éléments suivants :

- **#include <stdio.h>** est une directive de préprocesseur. Elle inclut la bibliothèque standard d'entrées-sorties.
- **int main()** définit la fonction principale, où commence l'exécution. Le type int indique qu'elle retourne un entier, généralement pour signaler le succès ou l'échec du programme.
- **printf("Hello, World!\n");** affiche un message à l'écran. La suite \n provoque un saut de ligne.
- **return 0;** termine la fonction et renvoie au système d'exploitation un code de succès.

1.3 Le processus de compilation

Le C est un langage compilé : le code source est transformé en un fichier exécutable que l'ordinateur peut lancer directement. La compilation comporte trois grandes étapes :

1. Le **préprocesseur** traite les directives comme #include et prépare le fichier source.
2. La **compilation** traduit ce code en code objet (fichier .o), une version binaire partielle du programme.
3. L'**édition des liens** relie les fichiers objets aux bibliothèques nécessaires et crée l'exécutable final.

Sur un système Unix ou Linux, on compile et on exécute un programme avec les commandes suivantes :

```
gcc -o mon_programme mon_programme.c
./mon_programme
```

La première commande compile le fichier mon_programme.c et produit un exécutable nommé mon_programme. La seconde lance cet exécutable depuis l'interpréteur de commandes.

GCC (GNU Compiler Collection) fait partie du projet GNU. Il ne faut pas confondre GNU et Linux : Linux fournit le noyau, tandis que GNU fournit de nombreux outils.

Outil GNU	Rôle
GCC	Compilateur C et C++
GDB	Débogueur
Make	Automatisation de la compilation
glibc	Bibliothèque standard du C

Outil GNU	Rôle
binutils	Assembleur et éditeur de liens (ld)
bash	Interpréteur de commandes

1.4 Configurer l'environnement de travail

Voici la marche à suivre avec Visual Studio Code (VS Code) sous Windows :

1. Téléchargez et installez VS Code depuis le site officiel : https://code.visualstudio.com
2. Installez un compilateur, car VS Code n'en contient pas. Sous Windows, MinGW (Minimalist GNU for Windows) est très utilisé : https://sourceforge.net/projects/mingw/files/Installer/
3. Ajoutez MinGW à la variable Path. Dans les paramètres système, ouvrez **Avancé**, puis **Variables d'environnement**. Dans les variables système, modifiez Path et ajoutez le chemin du dossier bin de MinGW, par exemple C:\mingw\bin.
4. Vérifiez l'installation en tapant gcc --version dans une invite de commandes.
5. Dans VS Code, ouvrez le panneau **Extensions**, recherchez **Code Runner**, puis installez cette extension. Elle permet d'exécuter un programme directement depuis l'éditeur.

Vous pouvez aussi tester vos programmes en ligne, sans rien installer, à l'adresse suivante : https://www.onlinegdb.com/online_c_compiler

1.5 Les commentaires

Les commentaires ne sont pas exécutés. Ils servent à expliquer ou à documenter le code. Le C en propose deux formes :

```
// Commentaire sur une seule ligne

/* Commentaire qui peut
   s'étendre sur plusieurs lignes */
```

Les commentaires multilignes ne peuvent pas être imbriqués. La première ligne ci-dessous est correcte, mais pas la seconde :

```
/* ceci est un commentaire correct */
/* ceci est /* évidemment */ incorrect */
```

1.6 Premier programme

Pour afficher « bonjour » à l'écran, il faut inclure la bibliothèque qui contient la fonction d'affichage, puis écrire la fonction main. La fonction printf, déclarée dans le fichier d'en-tête stdio.h, s'utilise ainsi : printf("ce que l'on veut écrire");

```
  #include <stdio.h>

  int main()
  {
       printf("bonjour\n"); /* toute instruction se termine par un point-virgule */
      return 0;
  }
```

Résultat à l'exécution :

```
bonjour
```

1.7 Les séquences d'échappement

La suite \n, à la fin de la chaîne, signifie un passage à la ligne (n pour *new line*). Le C dispose de plusieurs séquences de ce type pour contrôler l'affichage.

Séquence	Effet
\n	Nouvelle ligne
\t	Tabulation
\b	Retour arrière (*backspace*)
\r	Retour au début de la ligne, sans saut de ligne
\a	Signal sonore
\\	Affiche un antislash
\'	Affiche une apostrophe
\"	Affiche un guillemet double
\0	Caractère nul, qui marque la fin d'une chaîne

La tabulation permet d'aligner un affichage en colonnes :

```
#include <stdio.h>

int main() {
    printf("Nom\t\tAge\t\tVille\n");
    printf("Ali\t\t25\t\tErrachidia\n");
    printf("Hassan\t\t30\t\tRabat\n");
    return 0;
}
```

Résultat à l'exécution :

```
Nom             Age         Ville
Ali             25          Errachidia
Hassan          30          Rabat
```

La séquence \r ramène le curseur au début de la ligne. Dans l'exemple suivant, « Salut » écrase les cinq premières lettres de « Bonjour », et l'écran affiche donc **Salutur**.

```
#include <stdio.h>

int main() {
    printf("Bonjour\rSalut\n");
    return 0;
}
```

La séquence \b recule le curseur d'une position. Ci-dessous, il recule de quatre positions, jusqu'au J de « BONJOUR », puis « soir » remplace « JOUR ». L'écran affiche **BONsoir**.

```
#include <stdio.h>

int main() {
    printf("BONJOUR\b\b\b\bsoir");
    return 0;
}
```

Enfin, \a peut émettre un bip, selon le système utilisé :

```
#include <stdio.h>

int main() {
    printf("Attention\a\n");
    return 0;
}
```

1.8 Les types de base

Un programme manipule des variables et des constantes, dont il faut fixer le type. En mathématiques, on distingue notamment les entiers naturels (ℕ), les entiers relatifs (ℤ) et les

réels (ℝ), dont l'ordre de grandeur est illimité. Un ordinateur, lui, travaille en binaire avec une place limitée. Il dispose donc de deux grandes familles de types : les entiers et les rationnels.

Les entiers

Les tailles dépendent du compilateur et de la machine. Le tableau suivant donne les valeurs habituelles sur un système Linux 64 bits :

Type	Description	Taille	Valeur minimale	Valeur maximale
char	Caractère	1 octet	-128	127
short	Entier court	2 octets	-32 768	32 767
int	Entier standard	4 octets	-2 147 483 648	2 147 483 647
long	Entier long	8 octets	$-9{,}22 \times 10^{18}$	$9{,}22 \times 10^{18}$

Le préfixe **unsigned** réserve la variable aux valeurs positives et déplace le domaine :

Type	Valeur minimale	Valeur maximale
unsigned char	0	255
unsigned short	0	65 535
unsigned int	0	4 294 967 295
unsigned long	0	$1{,}84 \times 10^{19}$

Les limites exactes de votre système sont indiquées dans le fichier d'en-tête limits.h. Sur des systèmes plus anciens, un int peut n'occuper que 2 octets : il ne faut donc pas supposer sa taille sans la vérifier avec l'opérateur sizeof.

Les rationnels

Type	Précision	Chiffres significatifs	Plus petite valeur positive normale	Valeur maximale	Taille
float	Simple	6	$1{,}2 \times 10^{-38}$	$3{,}4 \times 10^{38}$	4 octets
double	Double	15	$2{,}2 \times 10^{-308}$	$1{,}8 \times 10^{308}$	8 octets
long double	Etendue	18 ou plus	$3{,}4 \times 10^{-4932}$	$1{,}1 \times 10^{4932}$	Variable

Les valeurs de long double correspondent au format courant de 80 bits sur processeur x86. Elles peuvent varier selon la plateforme.

Les booléens

Le C classique ne possède pas de type booléen dédié : tout type numérique peut exprimer une condition. La valeur 0 correspond à FAUX, et toute valeur différente de 0 correspond à VRAI.

1.9 La déclaration des variables

Déclarer une variable consiste à lui donner un nom et un type. Le compilateur sait alors quel genre de donnée elle va contenir.

```
int a;                  // Une variable entière
int b, c, d;            // Trois variables entières
float pi = 3.14;        // Une variable réelle initialisée
int e = 1, f = 2, g = 3; // Trois variables initialisées
```

Chapitre 2 : Les opérateurs standards

2.1 L'affectation

L'opérateur = attribue une valeur à une variable, selon la syntaxe nom_variable = expression;

```
pi = 3.1416;   // Affectation d'une constante
lettre = 'L';  // Affectation d'un caractère
a = b;         // La valeur de b est affectée à a
x = a + b;     // Le résultat de a + b est affecté à x
```

Le C propose aussi une écriture condensée. Au lieu de i = i + 2; on peut écrire i += 2; ce qui suit bien la logique : « on ajoute 2 à i ». Il existe de la même façon -=, *=, /= et %=.

2.2 Les opérateurs arithmétiques

Opérateur	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Modulo, c'est-à-dire le reste de la division entière (5 % 2 vaut 1)

2.3 Les opérateurs logiques

Opérateur	Signification	Résultat
&&	ET	Vrai si les deux opérandes sont vrais
||	OU	Vrai si au moins un opérande est vrai
!	NON	Vrai si l'opérande est faux

Le résultat est de type int : il vaut 0 pour FAUX et 1 pour VRAI.

```
32 && 2.3        donne 1
!65.43           donne 0
0 || !(32 > 12)  donne 0
```

2.4 Les opérateurs de comparaison

Ces opérateurs comparent deux valeurs. Leur résultat est très souvent utilisé dans les instructions conditionnelles.

Opérateur	Signification
==	Egal à
!=	Différent de
<	Inférieur à
<=	Inférieur ou égal à
>	Supérieur à
>=	Supérieur ou égal à

```
int a = 10, b = 5;
int result1 = a == b;  // 0, car a n'est pas égal à b
int result2 = a != b;  // 1, car a est différent de b
int result3 = a > b;   // 1, car a est supérieur à b
int result4 = a < b;   // 0, car a n'est pas inférieur à b
```

2.5 L'incrémentation et la décrémentation

Les opérateurs ++ et -- augmentent ou diminuent une variable de 1. Ainsi, i = i + 1 s'écrit i++ ou ++i, et i = i - 1 s'écrit i-- ou --i.

Si l'on modifie seulement la variable, par exemple dans une boucle, les deux notations sont équivalentes. Si l'on affecte en même temps sa valeur à une autre variable, leur position change le résultat :

Instruction	Effet
X = i++;	Copie d'abord i dans X, puis incrémente i
X = i--;	Copie d'abord i dans X, puis décrémente i
X = ++i;	Incrémente d'abord i, puis le copie dans X
X = --i;	Décrémente d'abord i, puis le copie dans X

```
N = 5;  X = N++;   Résultat : X = 5 et N = 6
N = 5;  X = ++N;   Résultat : X = 6 et N = 6
```

2.6 Les priorités des opérateurs

Le tableau suivant classe les opérateurs vus dans ce cours, du plus prioritaire au moins prioritaire :

Priorité	Opérateurs	Famille
1	( )	Parenthèses
2	! ++ --	Opérateurs unaires
3	* / %	Arithmétiques
4	+ -	Arithmétiques
5	< <= > >=	Comparaison
6	== !=	Comparaison
7	&&	Logique
8	||	Logique
9	= += -= *= /= %=	Affectation

Lorsque plusieurs opérateurs ont la même priorité, l'ordre dépend de leur associativité. Les opérateurs binaires, comme +, *, ou &&, s'évaluent de gauche à droite. Les opérateurs unaires et les opérateurs d'affectation s'évaluent de droite à gauche.

```
10 + 20 + 30 - 40 :
    10 + 20 donne 30
    30 + 30 donne 60
    60 - 40 donne 20

A *= B += 5, avec A = 3 et B = 4 :
    B += 5 donne B = 9
    A *= 9 donne A = 27
```

Chapitre 3 : Les conversions de type

3.1 Les conversions automatiques

Si un opérateur reçoit des opérandes de types différents, leurs valeurs sont automatiquement converties dans un type commun. Cette conversion se fait en général vers le type le plus large, afin de ne pas perdre de précision.

Lors d'une affectation, en revanche, la valeur de droite est convertie dans le type de la variable de gauche, ce qui peut entraîner une perte de précision.

```
int a = 5;
double b = 12.34;
float c;
c = a + b;
```

Pour calculer a + b, la variable a est convertie en double. Le résultat est ensuite converti en float pour être affecté à c, et il peut perdre de la précision à cette étape, car un float est moins précis qu'un double.

3.2 Les conversions lors de l'appel d'une fonction

Lors d'un appel, les arguments sont automatiquement convertis dans les types déclarés par la fonction. La fonction pow de math.h attend deux valeurs de type double et retourne un double :

```
int A = 8, Resultat;
Resultat = pow(A, 2);
```

Ici, A et 2 sont convertis en double. Le résultat, de type double, est ensuite converti en int pour être affecté à Resultat.

3.3 Une erreur classique

```
#include <stdio.h>

int main() {
    int A = 5, B = 2;
    float C;
    C = A / B;
    printf("La variable C est %f\n", C);
    return 0;
}
```

Résultat à l'exécution :

```
La variable C est 2.000000
```

Comme A et B sont des int, A / B est une division entière et vaut 2. Cette valeur est ensuite convertie en float, et C vaut 2.0 au lieu de 2.5. Pour obtenir le bon résultat, au moins un opérande doit être réel avant la division : C = (float)A / B;

3.4 Les règles de conversion dans une opération

Entre deux entiers. Les types char et short sont d'abord convertis en int. Si les deux opérandes n'ont pas encore le même type, le plus petit est converti vers le plus grand, selon l'échelle habituelle int, unsigned int, long, unsigned long. Si l'un est un int et l'autre un unsigned int, les deux sont donc traités comme des unsigned int.

Entre un entier et un rationnel. L'entier est converti dans le type du rationnel, afin que l'opération conserve la précision des nombres à virgule.

```
int a = 5;
float b = 2.5;
float c;
c = a + b; // a est converti en float
```

Entre deux rationnels. Le type le moins large est converti vers le plus large, selon l'échelle float, double, long double.

```
float a = 3.5;
double b = 4.5;
double c;
c = a + b; // a est converti en double
```

3.5 La conversion forcée

On peut convertir explicitement une valeur avec la syntaxe (type) expression. Le contenu de la variable ne change pas : seule la valeur utilisée dans le calcul est convertie.

```
int A = 4, B = 3;
float C;
C = (float)A / B;
```

Chapitre 4 : Lire et écrire des données

La bibliothèque stdio.h contient les fonctions qui assurent la communication du programme avec l'extérieur. Pour la lecture, on utilise surtout scanf (lecture formatée) et getchar (lecture d'un caractère). Pour l'écriture, on utilise printf (écriture formatée) et putchar (écriture d'un caractère).

4.1 L'écriture formatée avec printf

La fonction printf affiche du texte, des valeurs de variables ou des résultats d'expressions sur la sortie standard, c'est-à-dire l'écran.

```
printf("format", expression1, expression2, ...);
```

La chaîne de format peut contenir du texte, des séquences d'échappement et des spécificateurs de format. Il faut un spécificateur pour chaque expression à afficher.

```
#include <stdio.h>

int main() {
    int age = 25;
    float taille = 1.75;
    printf("L'age est %d ans et la taille est %.2f metres.\n", age,
taille);
    return 0;
}
```

Résultat à l'exécution :

```
L'age est 25 ans et la taille est 1.75 metres.
```

Les spécificateurs de format

Spécificateur	Type	Affichage
%d ou %i	int	Entier signé
%u	unsigned int	Entier non signé
%o	unsigned int	Entier en octal
%x	unsigned int	Entier en hexadécimal, en minuscules
%f	double	Réel en notation décimale
%e	double	Réel en notation scientifique
%g	double	Forme la plus concise, décimale ou scientifique
%c	char	Caractère
%s	Chaîne	Chaîne de caractères

Spécificateur	Type	Affichage
%ld ou %li	long	Entier long signé
%lu	unsigned long	Entier long non signé
%lo	unsigned long	Entier long en octal
%lx	unsigned long	Entier long en hexadécimal
%Lf	long double	Notation décimale
%Le	long double	Notation scientifique
%Lg	long double	Forme la plus concise

Avec printf, une valeur float est convertie en double au moment de l'appel. C'est pourquoi %f convient à la fois aux float et aux double.

La largeur et la précision

On peut indiquer une largeur minimale. Le nombre est alors aligné à droite dans l'espace réservé.

```
#include <stdio.h>

int main() {
    printf("%4d\n", 1);
    printf("%4d\n", 123);
    printf("%4d\n", 1234);
    return 0;
}
```

Résultat à l'exécution :

```
   1
 123
1234
```

Pour les réels, la syntaxe %n.mf fixe la largeur n du champ et le nombre m de décimales. La précision par défaut est de 6 décimales.

```
#include <stdio.h>

int main() {
    printf("%f\n", 12.34);
    printf("%10.3f\n", 100.123);
    printf("%10.2f\n", 100.123);
    return 0;
}
```

Résultat à l'exécution :

```
12.340000
   100.123
    100.12
```

4.2 La lecture formatée avec scanf

La fonction scanf lit des valeurs au clavier et les range dans des variables. Elle reçoit le format attendu, suivi des adresses des variables à remplir.

```
scanf("format", &variable1, &variable2, ...);
```

```
#include <stdio.h>

int main() {
    int age;
    float taille;
    printf("Entrez votre age : ");
    scanf("%d", &age);       // Lit un entier
    printf("Entrez votre taille (en metres) : ");
    scanf("%f", &taille);    // Lit un nombre réel
    printf("\nVotre age est : %d ans\n", age);
    printf("Votre taille est : %.2f metres\n", taille);
    return 0;
}
```

Résultat à l'exécution :

```
Entrez votre age : Entrez votre taille (en metres) :
Votre age est : 25 ans
Votre taille est : 1.75 metres
```

Le résultat ci-dessus correspond aux saisies 25 et 1.75.

Les spécificateurs de scanf ressemblent à ceux de printf, avec une différence importante : %f, %e et %g lisent un **float**, tandis que %lf, %le et %lg lisent un **double**. Pour les autres types, on retrouve notamment %d, %u, %o, %x, %c et %s.

Lire plusieurs valeurs

Une même instruction peut lire plusieurs variables. Lors de la saisie, une suite d'espaces, de tabulations ou de sauts de ligne compte comme un seul séparateur.

```
int jour, mois, annee;
scanf("%d %d %d", &jour, &mois, &annee);
```

Les saisies suivantes sont donc équivalentes :

```
24 11 1973
24   11   1973
24
11
1973
```

Si le format contient d'autres caractères, ceux-ci doivent être saisis exactement dans l'ordre indiqué.

```
int jour, mois, annee;
scanf("%d/%d/%d", &jour, &mois, &annee);
```

La saisie 24/11/1973 est acceptée, tandis que 24 11 1973 ne convient pas.

4.3 Lire et écrire un caractère

La fonction putchar affiche un caractère.

```
#include <stdio.h>

int main() {
    char c = 'A';
    putchar(c);
    putchar('\n');
    return 0;
}
```

Résultat à l'exécution :

```
A
```

La fonction **getchar** lit un caractère au clavier.

```
#include <stdio.h>

int main() {
    char c;
    printf("Entrez un caractere : ");
    c = getchar();
    printf("\nVous avez entre : ");
    putchar(c);
    putchar('\n');
    return 0;
}
```

Résultat à l'exécution :

```
Entrez un caractere :
Vous avez entre : Z
```

4.4 Ecrire un caractère dans un fichier

Pour écrire un caractère dans un fichier, on l'ouvre avec fopen, on écrit avec fputc, puis on le ferme avec fclose.

```
#include <stdio.h>

int main() {
    // Ouverture en écriture ; le fichier est créé s'il n'existe pas
    FILE *file = fopen("output.txt", "w");
    if (file == NULL) {                  // Vérifie si l'ouverture a
échoué
        printf("Erreur d'ouverture du fichier\n");
        return 1;                           // Signale une erreur
    }
    char c = 'C';
      fputc(c, file);                          // Ecrit le caractère
                          dans le fichier
   fclose(file);                      // Ferme le fichier et libère les
ressources
    printf("Le caractere '%c' a ete ecrit dans le fichier output.txt\
n", c);
    return 0;
}
```

Résultat à l'exécution :

```
  Le caractere 'C' a ete ecrit dans le fichier output.txt
```

Chapitre 5 : Les structures alternatives

Une structure alternative permet au programme de prendre une décision en fonction d'une condition. Les plus utilisées sont if et switch.

5.1 L'instruction if

L'instruction if exécute un bloc seulement si la condition est vraie.

```
if (condition) {
    // Code à exécuter si la condition est vraie
}
```

```
#include <stdio.h>

int main() {
    int nombre;
    printf("Entrez un nombre entier : ");
    scanf("%d", &nombre);
    printf("\n");
    if (nombre % 2 == 0) {
        printf("Le nombre %d est pair.\n", nombre);
    }
    printf("Programme termine.\n");
    return 0;
}
```

Résultat à l'exécution :

```
  Entrez un nombre entier :
  Le nombre 8 est pair.
  Programme termine.
```

5.2 L'instruction if...else

La partie else définit ce qu'il faut faire lorsque la condition est fausse.

```
#include <stdio.h>

int main() {
    int age = 16;
    if (age >= 18) {
        printf("Vous etes majeur.\n");
    } else {
        printf("Vous etes mineur.\n");
    }
    return 0;
}
```

Résultat à l'exécution :

```
  Vous etes mineur.
```

5.3 Les tests successifs

On enchaîne plusieurs else if pour tester des conditions successives. Le premier test vrai est exécuté, puis le programme sort de la structure.

```
#include <stdio.h>

int main() {
    float note;
    printf("Entrez une note : ");
    scanf("%f", &note);
    printf("\n");
    if (note >= 18) {
        printf("Excellent\n");
    } else if (note >= 16) {
        printf("T. Bien\n");
    } else if (note >= 14) {
        printf("Bien\n");
    } else if (note >= 12) {
        printf("A. Bien\n");
    } else if (note >= 10) {
        printf("Passable\n");
    } else {
        printf("Rattrapage\n");
    }
    return 0;
}
```

Résultat à l'exécution :

```
Entrez une note :
Bien
```

5.4 A quel if se rattache un else ?

Un else est toujours rattaché au dernier if qui n'en possède pas. Dans le premier extrait, il dépend donc de if (A > B). Dans le second, les accolades le rattachent au premier if.

```
if (N > 0)
    if (A > B)
        MAX = A;
    else
        MAX = B;

if (N > 0) {
    if (A > B)
        MAX = A;
} else
    MAX = B;
```

Le programme suivant compare deux nombres :

```
#include <stdio.h>

int main() {
    int A, B;
    printf("Entrer deux nombres entiers :\n");
    scanf("%d %d", &A, &B);
    if (A > B)
        printf("%d est plus grand que %d\n", A, B);
    else if (A < B)
        printf("%d est plus petit que %d\n", A, B);
    else
        printf("%d est egal a %d\n", A, B);
    return 0;
}
```

Résultat à l'exécution :

```
Entrer deux nombres entiers :
7 est plus grand que 3
```

Le test peut aussi porter sur un caractère :

```
#include <stdio.h>

int main() {
    char C;
    printf("Entrez une commande (O ou N) : ");
    C = getchar();
    if (C == 'O') {
        printf("Commande acceptee : Option O\n");
    } else if (C == 'N') {
        printf("Au revoir...\n");
    } else {
        printf("Erreur d'entree\n");
    }
    return 0;
}
```

Selon la saisie, le programme affiche :

Saisie	Affichage
O	Commande acceptee : Option O
N	Au revoir...
X, ou tout autre caractère	Erreur d'entree

5.5 L'instruction switch

L'instruction switch choisit parmi plusieurs cas selon la valeur d'une variable entière ou d'un caractère.

```
switch (variable) {
    case valeur1:
        // Code si variable == valeur1
        break;
    case valeur2:
        // Code si variable == valeur2
        break;
    default:
        // Code si aucune valeur ne correspond
        break;
}
```

L'instruction break arrête le traitement du switch. Sans elle, l'exécution continuerait dans le cas suivant.

```
#include <stdio.h>

int main() {
    int jour;
    printf("Entrez un numero (1 a 7) pour un jour de la semaine : ");
    scanf("%d", &jour);
    printf("\n");
    switch (jour) {
        case 1: printf("Lundi\n"); break;
        case 2: printf("Mardi\n"); break;
        case 3: printf("Mercredi\n"); break;
        case 4: printf("Jeudi\n"); break;
        case 5: printf("Vendredi\n"); break;
        case 6: printf("Samedi\n"); break;
        case 7: printf("Dimanche\n"); break;
        default: printf("Jour inconnu\n"); break;
    }
    return 0;
}
```

Résultat à l'exécution :

```
Entrez un numero (1 a 7) pour un jour de la semaine :
Mercredi
```

Le même traitement peut s'écrire avec if...else if, mais switch est souvent plus lisible lorsqu'on compare une seule variable à des valeurs précises :

```
if (jour == 1) {
    printf("Lundi\n");
} else if (jour == 2) {
    printf("Mardi\n");
} else {
    printf("Autre jour\n");
}
```

5.6 Exercice d'application

Enoncé. Ecrire un programme qui affiche un menu de boissons et confirme le choix de l'utilisateur :

1. avec l'instruction switch ;
2. avec l'instruction if...else.

Corrigé avec switch

```
#include <stdio.h>

int main() {
    int choix;
    printf("Menu des boissons :\n");
    printf("1. Cafe\n");
    printf("2. The\n");
    printf("3. Jus d'orange\n");
    printf("4. Eau\n");
    printf("Entrez votre choix (1-4) : ");
    scanf("%d", &choix);
    printf("\n");
    switch (choix) {
        case 1: printf("Vous avez choisi : Cafe\n"); break;
        case 2: printf("Vous avez choisi : The\n"); break;
        case 3: printf("Vous avez choisi : Jus d'orange\n"); break;
        case 4: printf("Vous avez choisi : Eau\n"); break;
       default: printf("Choix invalide, veuillez choisir entre 1 et
4.\n"); break;
    }
    return 0;
}
```

Résultat à l'exécution :

```
Menu des boissons :
1. Cafe
2. The
3. Jus d'orange
4. Eau
Entrez votre choix (1-4) :
Vous avez choisi : The
```

Corrigé avec if...else

Seule la partie du choix change par rapport au corrigé précédent :

```
if (choix == 1) {
    printf("Vous avez choisi : Cafe\n");
} else if (choix == 2) {
    printf("Vous avez choisi : The\n");
} else if (choix == 3) {
    printf("Vous avez choisi : Jus d'orange\n");
} else if (choix == 4) {
    printf("Vous avez choisi : Eau\n");
} else {
    printf("Choix invalide, veuillez choisir entre 1 et 4.\n");
}
```

Chapitre 6 : Les structures répétitives

Les boucles exécutent un bloc d'instructions plusieurs fois, tant qu'une condition est remplie.

6.1 La boucle while

La boucle while, « tant que », teste la condition avant chaque tour. Si la condition est fausse dès le départ, le bloc n'est jamais exécuté.

```
while (condition) {
    // Instructions à répéter
}
```

```
#include <stdio.h>

int main() {
    int compteur = 0;
    while (compteur < 5) {
        printf("%d\n", compteur);
        compteur++;
    }
    return 0;
}
```

Résultat à l'exécution :

```
0
1
2
3
4
```

Lorsque le bloc ne contient qu'une seule instruction, les accolades ne sont pas obligatoires. La position de ++ ou de -- influence alors l'affichage :

Code (avec int i)	Valeur initiale	Affichage
while (i < 4) printf("%d\n", i++);	i = 0	0 1 2 3
while (i < 4) printf("%d\n", ++i);	i = 0	1 2 3 4
while (i) printf("%d\n", i--);	i = 4	4 3 2 1

Dans la dernière ligne, while (i) signifie « tant que i est différent de zéro ». L'opérateur i-- fournit d'abord la valeur à printf, puis décrémente i.

6.2 La boucle do...while

La boucle do...while, « faire, tant que », teste la condition après le bloc. Celui-ci est donc toujours exécuté au moins une fois.

```
do {
    // Instructions à répéter
} while (condition);
```

```
#include <stdio.h>

int main() {
    int compteur = 0;
    do {
        printf("Compteur : %d\n", compteur);
        compteur++;
    } while (compteur < 5);
    return 0;
}
```

Résultat à l'exécution :

```
Compteur : 0
Compteur : 1
Compteur : 2
Compteur : 3
Compteur : 4
```

Elle convient très bien au contrôle d'une saisie. Le programme suivant redemande un nombre tant qu'il ne se trouve pas entre 1 et 10 :

```
#include <stdio.h>

int main() {
    int n;
    do {
        printf("Entrer un nombre entre 1 et 10 :\n");
        scanf("%d", &n);
    } while (n < 1 || n > 10);
    printf("Le nombre %d est valide.\n", n);
    return 0;
}
```

Résultat à l'exécution :

```
Entrer un nombre entre 1 et 10 :
Entrer un nombre entre 1 et 10 :
Le nombre 7 est valide.
```

Le résultat ci-dessus correspond aux saisies 15, puis 7.

6.3 La boucle for

La boucle for est utilisée lorsque le nombre de tours est connu à l'avance. Elle rassemble l'initialisation, la condition et la mise à jour.

```
for (initialisation; condition; mise à jour) {
    // Instructions à répéter
}
```

```
#include <stdio.h>

int main() {
    for (int i = 0; i < 5; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}
```

Résultat à l'exécution :

```
i = 0
i = 1
i = 2
i = 3
i = 4
```

Le programme suivant affiche les carrés des nombres de 1 à 4 :

```
#include <stdio.h>

int main() {
    int i;
    for (i = 1; i <= 4; i++) {
        printf("Le carre de %d est %d\n", i, i * i);
    }
    return 0;
}
```

Résultat à l'exécution :

```
Le carre de 1 est 1
Le carre de 2 est 4
Le carre de 3 est 9
Le carre de 4 est 16
```

6.4 Quelle boucle choisir ?

Boucle	A utiliser lorsque...
for	Le nombre de tours est connu ou fixe.
while	Le nombre de tours n'est pas connu à l'avance, mais une condition détermine la fin.
do...while	Le bloc doit être exécuté au moins une fois.

Chapitre 7 : Les tableaux

Un tableau permet de stocker plusieurs éléments du même type en mémoire, sous un même nom.

7.1 Les tableaux à une dimension

Déclaration

On déclare un tableau statique en donnant le type de ses éléments, son nom et sa taille : type nomTableau[taille];

```
int nombres[5];     // Tableau de 5 entiers
float notes[10];    // Tableau de 10 réels
char lettres[26];   // Tableau de 26 caractères
```

Initialisation

Un tableau peut être initialisé dès sa déclaration. Si toutes les valeurs ne sont pas fournies, les éléments restants valent zéro, ou \0 pour les caractères. Si la taille est omise, le compilateur la déduit du nombre de valeurs.

```
int nombres[5] = {1, 2, 3, 4, 5};
float notes[3] = {12.5, 15.0, 9.8};
char voyelles[5] = {'a', 'e', 'i', 'o', 'u'};
int partiel[5] = {1, 2};          // Les trois derniers éléments valent
0
int implicite[] = {1, 2, 3, 4, 5};  // Taille déduite : 5
```

Accès aux éléments

On accède à un élément par son indice, et **le premier indice est 0**.

```
  nombres[0] = 10;             // Modifie le premier élément
  printf("%d\n", nombres[0]);  // Affiche 10
```

Parcours d'un tableau

On parcourt généralement un tableau avec une boucle for.

```
  #include <stdio.h>

  int main() {
      int A[5] = {1, 2, 3, 4, 5};
      int i;
      for (i = 0; i < 5; i++)
          printf("%3d", A[i]);
      printf("\n");
      return 0;
  }
```

Résultat à l'exécution :

```
  1  2  3  4  5
```

Pour remplir un élément au clavier, on donne son adresse, &A[i], comme pour une variable ordinaire.

```
#include <stdio.h>

int main() {
    int A[5];
    int i;
    for (i = 0; i < 5; i++)
        scanf("%d", &A[i]);
    printf("Les nouvelles valeurs introduites sont\n");
    for (i = 0; i < 5; i++)
        printf("%d ", A[i]);
    printf("\n");
    return 0;
}
```

Résultat à l'exécution :

```
Les nouvelles valeurs introduites sont
6 7 8 9 10
```

7.2 Les tableaux à deux dimensions

Un tableau à deux dimensions représente souvent une matrice. Chaque élément est repéré par l'indice de sa ligne, puis par celui de sa colonne.

```
int matrice[3][4];  // 3 lignes et 4 colonnes

int m1[2][3] = {
    {1, 2, 3},      // Première ligne
    {4, 5, 6}       // Deuxième ligne
};

int m2[][3] = {     // Nombre de lignes déduit par le compilateur
    {1, 2, 3},
    {4, 5, 6}
};
```

Seule la première dimension peut être omise : toutes les autres doivent être indiquées.

```
#include <stdio.h>

int main() {
    int matrice[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("%d\n", matrice[0][1]);  // Ligne 1, colonne 2
    matrice[0][1] = 10;
    printf("La nouvelle valeur est : %d\n", matrice[0][1]);
    return 0;
}
```

Résultat à l'exécution :

```
2
La nouvelle valeur est : 10
```

On parcourt une matrice avec deux boucles imbriquées :

```
#include <stdio.h>

int main() {
    int A[3][2] = {{1, 2}, {10, 20}, {100, 200}};
    int i, j;
    printf("Affichage de la matrice :\n");
    for (i = 0; i < 3; i++) {       // Parcours des lignes
        for (j = 0; j < 2; j++) {   // Parcours des colonnes
            printf("%5d", A[i][j]);
        }
        printf("\n");
    }
    int ligne = 1, colonne = 0;
    printf("\nElement [%d][%d] = %d\n", ligne, colonne, A[ligne][colonne]);
    return 0;
}
```

Résultat à l'exécution :

```
Affichage de la matrice :
    1    2
   10   20
  100  200

Element [1][0] = 10
```

7.3 Application

Enoncé. Ecrire un programme qui saisit les valeurs d'une matrice de 3 lignes et 2 colonnes, l'affiche, puis affiche l'élément dont l'utilisateur donne les indices.

Corrigé

```
#include <stdio.h>

int main() {
    int A[3][2];
    int i, j;
    printf("Saisissez les valeurs pour une matrice 3x2 :\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 2; j++) {
            printf("Element [%d][%d] : ", i, j);
            scanf("%d", &A[i][j]);
        }
    }
    printf("\nAffichage de la matrice :\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 2; j++) {
            printf("%5d", A[i][j]);
        }
        printf("\n");
    }
    int ligne, colonne;
    printf("\nEntrez l'indice de la ligne (0-2) : ");
```

```
    scanf("%d", &ligne);
    printf("Entrez l'indice de la colonne (0-1) : ");
    scanf("%d", &colonne);
    printf("\nElement [%d][%d] = %d\n", ligne, colonne, A[ligne][colonne]);
    return 0;
}
```

Résultat à l'exécution :

```
Saisissez les valeurs pour une matrice 3x2 :
Element [0][0] : Element [0][1] : Element [1][0] : Element [1][1] :
Element [2][0] : Element [2][1] :
Affichage de la matrice :
    1    2
    3    4
    5    6

Entrez l'indice de la ligne (0-2) : Entrez l'indice de la colonne (0-1)
:
Element [2][1] = 6
```

Le résultat ci-dessus correspond aux valeurs 1 à 6, puis aux indices 2 et 1. Comme les saisies ne s'affichent pas ici, les invites apparaissent à la suite.

Chapitre 8 : Les chaînes de caractères

En C, une chaîne de caractères est un tableau de caractères à une dimension, terminé par le caractère nul \0, qui en marque la fin.

8.1 Déclaration et initialisation

Lors de la déclaration, on indique la place à réserver, en comptant le caractère final : char nom[longueur];

```
  char TXT[9] = "BONJOUR!";
```

Cette chaîne compte 8 caractères visibles et occupe 9 cases :

Indice	0	1	2	3	4	5	6	7	8
Contenu	B	O	N	J	O	U	R	!	\0

Le nom TXT représente l'adresse du premier caractère. D'autres écritures sont possibles :

```
  char chaine1[] = "Bonjour";     // Taille déterminée automatiquement
  char chaine2[10] = "Salut";      // Taille fixée manuellement
  char chaine3[10] = {'H', 'e', 'l', 'l', 'o', '\0'};  // \0 écrit
explicitement
```

8.2 Les chaînes constantes

Une chaîne constante s'écrit entre guillemets, et la chaîne vide s'écrit "". Elle peut contenir des séquences d'échappement.

```
  #include <stdio.h>

  int main() {
      char C[] = "Ce texte\n s'ecrira sur\n 3 lignes";
      printf("%s\n", C);
      return 0;
  }
```

Résultat à l'exécution :

```
  Ce texte
   s'ecrira sur
   3 lignes
```

Pour placer une apostrophe dans une constante de caractère, on écrit '\''.

```
  #include <stdio.h>

  int main() {
      char txt[] = {'L', '\'', 'a', 's', 't', 'u', 'c', 'e', '\0'};
      printf("%s\n", txt);
      return 0;
  }
```

Résultat à l'exécution :

```
L'astuce
```

Le spécificateur %s affiche la chaîne jusqu'au premier \0 rencontré. Ainsi, la chaîne "Bonjour\0Monde" s'affiche seulement sous la forme Bonjour.

Plusieurs chaînes constantes séparées par des espaces sont réunies à la compilation.

```
#include <stdio.h>

int main() {
    char TXT[] = "un " "deux " "trois";
    printf("%s\n", TXT);
    return 0;
}
```

Résultat à l'exécution :

```
un deux trois
```

Il ne faut pas confondre 'x' et "x". 'x' est un caractère, de valeur numérique 120, codé sur 1 octet. "x" est une chaîne, qui contient 'x' et '\0', et occupe 2 octets.

8.3 La représentation en mémoire

Déclaration	Contenu en mémoire	Remarque
char txt[] = "hello";	h e l l o \0	6 octets réservés automatiquement
char txt[6] = "hello";	h e l l o \0	Taille exacte
char txt[8] = "hello";	h e l l o \0 \0 \0	Les cases restantes valent \0
char txt[5] = "hello";	h e l l o	Pas de \0 : ce n'est pas une chaîne valide pour %s, puts ou strlen
char txt[4] = "hello";	h e l l	Texte trop long : le compilateur signale un problème, à corriger

8.4 L'accès aux éléments

On accède aux caractères par leur indice, comme dans tout tableau.

```
#include <stdio.h>

int main() {
    char txt[] = "hello";
    printf("Premier caractere : %c\n", txt[0]);
    printf("Deuxieme caractere : %c\n", txt[1]);
    printf("Cinquieme caractere : %c\n", txt[4]);
    return 0;
}
```

Résultat à l'exécution :

```
Premier caractere : h
Deuxieme caractere : e
Cinquieme caractere : o
```

Ici, txt[5] contient le caractère nul \0, qui n'a pas de représentation visible.

8.5 Les fonctions de stdio.h

L'affichage

La fonction printf utilise le spécificateur %s et permet de fixer une largeur. Un nombre positif aligne le texte à droite, un nombre négatif à gauche.

```
#include <stdio.h>

int main() {
    char NOM[] = "Hello world";
    printf(":%s:\n", NOM);
    printf(":%15s:\n", NOM);
    printf(":%-15s:\n", NOM);
    return 0;
}
```

Résultat à l'exécution :

```
:Hello world:
:    Hello world:
:Hello world    :
```

La fonction puts affiche simplement une chaîne, suivie d'un saut de ligne. L'appel puts(ch) équivaut donc à printf("%s\n", ch). Elle est plus simple, mais printf reste plus souple pour formater l'affichage.

Attention : printf avec %s et puts ont besoin du caractère \0. Le code suivant a un comportement indéfini, car ces fonctions peuvent continuer à lire la mémoire jusqu'à rencontrer un \0 par hasard :

```
char chaine[] = {'H', 'e', 'l', 'l', 'o'};  // Pas de '\0'
puts(chaine);                               // A ne pas faire
```

La lecture avec scanf

La fonction scanf avec %s lit une chaîne jusqu'au premier espace, tabulation ou saut de ligne. Elle ne convient donc pas à un texte contenant des espaces. Le nom de la chaîne représentant déjà une adresse, on n'écrit pas de & devant lui. Enfin, un simple %s ne protège pas du débordement : on préférera une largeur, comme %99s pour un tableau de 100 caractères.

```c
#include <stdio.h>

int main() {
    char chaine[100];
    printf("Entrez une chaine : ");
    scanf("%99s", chaine);
    printf("\nVous avez entre : %s\n", chaine);
    return 0;
}
```

Résultat à l'exécution :

```
Entrez une chaine :
Vous avez entre : Bonjour
```

La fonction gets, à éviter

La fonction gets lisait une ligne entière, espaces compris, sans contrôler la taille du tableau. Elle provoquait ainsi des débordements de mémoire et a été retirée de la norme du langage. Elle ne doit plus être utilisée : il faut lui préférer fgets.

La lecture avec fgets

La fonction fgets lit une ligne en limitant le nombre de caractères.

```c
#include <stdio.h>

int main() {
    char chaine[10];
    printf("Entrez une chaine : ");
    fgets(chaine, 10, stdin);  // Au plus 9 caractères, plus '\0'
    printf("\nVous avez entre : %s", chaine);
    return 0;
}
```

Résultat à l'exécution :

```
Entrez une chaine :
Vous avez entre : Salut
```

Ses trois arguments sont le tableau de destination, sa taille et le flux d'entrée, ici stdin pour le clavier. Si le saut de ligne tient dans le tableau, fgets le conserve dans la chaîne.

8.6 Les fonctions de string.h

Fonction	Rôle
strlen(s)	Donne la longueur de s, sans compter le \0
strcpy(dest, src)	Copie src dans dest
strcat(dest, src)	Ajoute src à la fin de dest
strcmp(s1, s2)	Compare s1 et s2

Pour strcpy et strcat, le tableau de destination doit être assez grand. La fonction strcmp retourne 0 si les chaînes sont égales, une valeur négative si s1 précède s2, et une valeur positive sinon.

```
#include <stdio.h>
#include <string.h>

int main() {
    char txt[] = "Bonjour";
    char copie[10];
    char phrase[30] = "Bonjour";
    char fin[] = " tout le monde";

    printf("Longueur : %zu\n", strlen(txt));
    strcpy(copie, txt);
    printf("Copie : %s\n", copie);
    strcat(phrase, fin);
    printf("Phrase : %s\n", phrase);
    printf("Comparaison avec Bonjour : %d\n", strcmp(txt, "Bonjour"));
    printf("Bonjour est avant Bonsoir : %d\n", strcmp(txt, "Bonsoir") < 0);
    return 0;
}
```

Résultat à l'exécution :

```
Longueur : 7
Copie : Bonjour
Phrase : Bonjour tout le monde
Comparaison avec Bonjour : 0
Bonjour est avant Bonsoir : 1
```

8.7 Les conversions entre chaînes et nombres

De la chaîne vers le nombre

La bibliothèque stdlib.h fournit trois fonctions :

Fonction	Conversion
atoi(s)	Chaîne vers int
atol(s)	Chaîne vers long
atof(s)	Chaîne vers double

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char s1[] = "123";
    char s2[] = "123456789";
    char s3[] = "123.456";
    int x = atoi(s1);
    long y = atol(s2);
    double z = atof(s3);
    printf("x + 2 = %d\n", x + 2);
    printf("y = %ld\n", y);
    printf("z = %.3f\n", z);
    return 0;
}
```

Résultat à l'exécution :

```
x + 2 = 125
y = 123456789
z = 123.456
```

Ces fonctions ignorent les espaces au début de la chaîne et s'arrêtent au premier caractère non convertible. Elles retournent zéro si la chaîne n'est pas convertible. Elles ne contrôlent pas le domaine du résultat.

Du nombre vers la chaîne

Certains compilateurs, notamment sous Windows, proposent itoa(n, s, b) pour un int et ltoa(n, s, b) pour un long. Le paramètre n est le nombre à convertir, s la chaîne de destination, et b la base utilisée, par exemple 2, 10 ou 16.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 1234;
    char s[20];
    itoa(n, s, 2);  // Conversion en base 2
    printf("Le nombre %d en binaire est : %s\n", n, s);
    return 0;
}
```

Ces fonctions ne font toutefois pas partie de la norme du C : elles sont par exemple absentes de la plupart des environnements Unix et Linux. La solution portable est sprintf, qui écrit dans une chaîne à la manière de printf.

```
#include <stdio.h>

int main() {
    int n = 1234;
    char s[20];
    sprintf(s, "%d", n);
    printf("Le nombre %d en chaine de caracteres est : %s\n", n, s);
    return 0;
}
```

Résultat à l'exécution :

```
Le nombre 1234 en chaine de caracteres est : 1234
```

8.8 Les tableaux de chaînes

Un tableau de chaînes est un tableau à deux dimensions : le nombre de chaînes, puis la longueur maximale de chacune, en comptant le \0.

```
char jours[7][9] = {"Lundi", "Mardi", "Mercredi", "Jeudi",
                    "Vendredi", "Samedi", "Dimanche"};
```

Ce tableau comporte 7 lignes et 9 colonnes. Chaque jour peut compter au plus 8 caractères, et la dernière case de « Mercredi », « Vendredi » et « Dimanche » contient le \0. Dans les noms plus courts, toutes les cases après le texte valent \0.

On accède à un caractère avec deux indices, et à une chaîne complète avec un seul :

```
#include <stdio.h>

int main() {
    char jours[7][9] = {"Lundi", "Mardi", "Mercredi", "Jeudi",
                        "Vendredi", "Samedi", "Dimanche"};
    printf("Premier caractere du premier jour : %c\n", jours[0][0]);
     printf("Deuxieme caractere du deuxieme jour : %c\n", jours[1]
[1]);
    printf("Premier jour : %s\n", jours[0]);
    printf("Dernier jour : %s\n", jours[6]);
    return 0;
}
```

Résultat à l'exécution :

```
Premier caractere du premier jour : L
Deuxieme caractere du deuxieme jour : a
Premier jour : Lundi
Dernier jour : Dimanche
```

On peut aussi parcourir tout le tableau. Le test jours[i][j] != '\0' arrête la lecture de chaque mot à sa fin.

```
#include <stdio.h>

int main() {
    char jours[7][9] = {"Lundi", "Mardi", "Mercredi", "Jeudi",
                        "Vendredi", "Samedi", "Dimanche"};
    for (int i = 0; i < 7; i++) {
        for (int j = 0; jours[i][j] != '\0'; j++) {
            printf("%c", jours[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Résultat à l'exécution :

```
Lundi
Mardi
Mercredi
Jeudi
Vendredi
Samedi
Dimanche
```

Chapitre 9 : Les fonctions

Une fonction est un sous-programme qui exécute un ensemble d'instructions. Elle peut être appelée plusieurs fois, à différents endroits, par main ou par une autre fonction.

9.1 La déclaration

Une fonction doit être déclarée avant son utilisation. Cette déclaration, appelée **prototype**, en définit le profil :

- le **type de retour**, c'est-à-dire le type de la valeur renvoyée (void si la fonction ne renvoie rien, on parle alors de procédure) ;
- le **nom** de la fonction ;
- les **paramètres**, de zéro à n, chacun avec un type et un nom.

```
float moyenne(int a, int b);
```

Ce prototype indique au compilateur que moyenne prend deux paramètres de type int et retourne une valeur de type float.

9.2 L'implémentation

Une fonction déclarée doit ensuite être implémentée, en reprenant exactement la même signature.

```
#include <stdio.h>

float moyenne(int a, int b);  // Déclaration

float moyenne(int a, int b) { // Implémentation
    float m;
    m = (a + b) / 2.0;        // 2.0 impose une division réelle
    return m;
}

int main() {
    int x = 10, y = 20;
    printf("La moyenne de %d et %d est %.2f\n", x, y, moyenne(x, y));
    return 0;
}
```

Résultat à l'exécution :

```
La moyenne de 10 et 20 est 15.00
```

9.3 L'appel d'une fonction

Une fonction peut être appelée depuis main ou depuis une autre fonction.

```
#include <stdio.h>
```

```
float moyenne(int a, int b);
void afficherMoyenne(int x, int y);

float moyenne(int a, int b) {
    return (a + b) / 2.0;
}

void afficherMoyenne(int x, int y) {
    float m = moyenne(x, y);  // Appel de moyenne
    printf("La moyenne de %d et %d est : %.2f\n", x, y, m);
}

int main() {
    int a = 10, b = 20;
     afficherMoyenne(a, b);     // afficherMoyenne appelle elle-même
moyenne
    return 0;
}
```

Résultat à l'exécution :

```
La moyenne de 10 et 20 est : 15.00
```

Une fonction peut aussi être définie après main, à condition que son prototype figure avant l'appel :

```
#include <stdio.h>

float moyenne(int a, int b);  // Prototype

int main() {
    int a = 10, b = 20;
    printf("La moyenne de %d et %d est : %.2f\n", a, b, moyenne(a, b));
    return 0;
}

float moyenne(int a, int b) { // Définition après main
    return (a + b) / 2.0;
}
```

Résultat à l'exécution :

```
La moyenne de 10 et 20 est : 15.00
```

Lorsque la fonction se termine, l'exécution reprend immédiatement après l'endroit où elle a été appelée. C'est le **retour de fonction**, illustré par le schéma suivant.

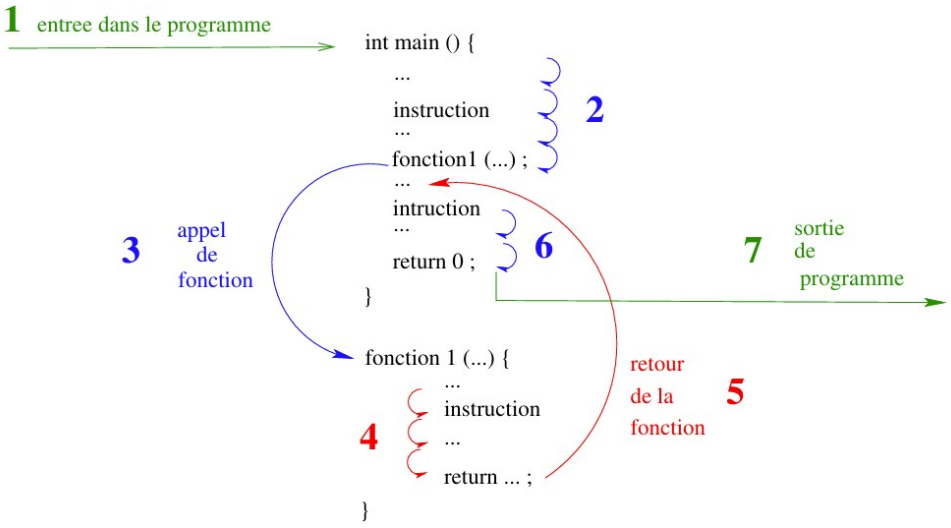


Figure 1 : déroulement de l'appel et du retour d'une fonction

9.4 La portée des variables

Les variables déclarées dans une fonction, y compris ses paramètres, sont **locales** à cette fonction : elles n'existent et ne sont utilisables que dans celle-ci. Chaque fonction possède son propre espace mémoire, ce qui permet d'utiliser le même nom dans plusieurs fonctions sans conflit.

```
#include <stdio.h>

float moyenne(int a, int b);

int main() {
    float m;
    float n;
    int a;
    int a2;
    a = 5;
    a2 = 8;
    m = 3.1515;
    // Instant 1
    n = moyenne(a, a2);
    // Instant 4
    printf("a=%d, a2=%d, m=%f, n=%f\n", a, a2, m, n);
    return 0;
}

float moyenne(int a, int b) {
    float m;
    int s;
    // Instant 2
    s = a + b;
    m = s / 2.0;
    a = 42;
    // Instant 3
    printf("a=%d, b=%d, m=%f, s=%d\n", a, b, m, s);
    return m;
}
```

Résultat à l'exécution :

```
a=42, b=8, m=6.500000, s=13
a=5, a2=8, m=3.151500, n=6.500000
```

Le tableau suivant décrit l'état de la mémoire aux quatre instants indiqués dans le programme :

Instant	Variables de main	Variables de moyenne
1 : avant l'appel	a = 5, a2 = 8, m = 3.1515, n non initialisée	Elles n'existent pas encore
2 : début de moyenne	Inchangées	a = 5, b = 8 (copies), m et s non initialisées
3 : après les calculs	Inchangées	a = 42, b = 8, m = 6.5, s = 13
4 : retour dans main	a = 5, a2 = 8, m = 3.1515, n = 6.5	Elles n'existent plus

Trois points sont à retenir :

1. La modification de **a** dans moyenne n'affecte pas le **a** de main, car les paramètres sont **passés par valeur**.
2. Chaque fonction possède ses propres variables locales.
3. Les variables locales de moyenne ne sont plus accessibles lorsque la fonction se termine.

9.5 Les règles à respecter

Domaine	Règle
Signature	Le prototype et l'implémentation doivent avoir le même type de retour, le même nombre de paramètres et les mêmes types.
Déclaration	Toute fonction doit être déclarée avant son utilisation, par un prototype ou par son implémentation directe.
Implémentation	Une fonction déclarée et appelée doit être implémentée.
Appel	Les arguments doivent correspondre en nombre et en type à ceux attendus.
Retour	La valeur retournée doit être utilisée de façon cohérente avec son type.

9.6 Exercice d'application

Enoncé. Ecrire une fonction puissance, qui prend deux paramètres entiers (la valeur et l'exposant) et retourne la valeur élevée à cet exposant. Par exemple, puissance(2, 3) retourne 8. La fonction main doit demander les deux nombres à l'utilisateur, puis appeler puissance.

Corrigé

```
#include <stdio.h>

int puissance(int valeur, int exposant) {
    int resultat = 1;
    int i;
    for (i = 0; i < exposant; i++) {
        resultat = resultat * valeur;
    }
    return resultat;
}

int main() {
    int v, e, resultat;
    printf("Rentrez une valeur : ");
    scanf("%d", &v);
    printf("Rentrez l'exposant : ");
    scanf("%d", &e);
    resultat = puissance(v, e);
    printf("\n%d puissance %d vaut %d\n", v, e, resultat);
    return 0;
}
```

Résultat à l'exécution :

```
Rentrez une valeur : Rentrez l'exposant :
2 puissance 3 vaut 8
```

Voici le déroulement des itérations pour 2 puissance 3 :

Itération	resultat avant	Multiplication	resultat après
i = 0	1	1 × 2	2
i = 1	2	2 × 2	4
i = 2	4	4 × 2	8

On peut aussi utiliser la fonction pow de math.h, qui retourne un double. Sous Linux, il faut alors ajouter l'option -lm à la compilation.

```
#include <stdio.h>
#include <math.h>

int main() {
    int v, e;
    double resultat;
    printf("Rentrez une valeur : ");
    scanf("%d", &v);
    printf("Rentrez l'exposant : ");
    scanf("%d", &e);
    resultat = pow(v, e);
    printf("\n%d puissance %d vaut %.0f\n", v, e, resultat);
    return 0;
}
```

Résultat à l'exécution :

```
Rentrez une valeur : Rentrez l'exposant :
2 puissance 3 vaut 8
```